

Practice Lab 09 — Inventory Dashboard

Scenario

A small shop needs a simple internal dashboard to see what's in their inventory. Your team is building it. This is a completely new project, in a new domain — you are not extending anything from class.

Timebox

Approximately 3 hours, structured like this:

- **0–15 min:** read these requirements as a team, plan your approach (see "Team plan" below).
- **~15–120 min:** build CORE.
- **~120–175 min:** test, polish CSS/responsiveness, fix bugs, then attempt optional extras with whatever time remains.
- **Last 5–10 min:** complete the checklist and team reflection below.

The instructor may not be available during this block — everything you need is in this document.

Local setup

Start from nothing. Create a brand-new local Vite React + TypeScript project yourselves, the same way you did in Session 05 — do not start from any provided project or template. No component files, no starter code, and no TaskFlow code are provided.

Provided data

You are given one file: `assets/products.json`. Copy it into your new project so your app can fetch it through the Vite dev server (think about where a static file needs to live so `fetch()` can reach it — you've already learned how the dev server serves files). Each product in the file looks like this:

```
{ "id": 1, "name": "Wireless Mouse", "category": "Electronics", "price": 24.99, "inStock": true }
```

CORE requirements (required)

Build an inventory dashboard where:

1. When the app loads, it fetches the product list and shows a loading indicator while waiting.
2. If the fetch fails, a clear error message appears with a way to try again.
3. Once loaded, every product is displayed, showing at least its name, category, price, and whether it's in stock.
4. The dashboard shows one meaningful summary number calculated from the product list — for example, how many products exist in total. (You decide exactly what to show.)
5. Users can add a new product to the list, providing at least a name, a category, and a price.
6. Users can mark a product as in-stock or out-of-stock.
7. Users can remove a product from the list entirely.

That's the whole required list. Get all of it working and tested before spending time on anything below.

Optional extras (attempt only after CORE is solid, in whatever order your team prefers)

None are required. Pick as many or as few as your remaining time allows:

- Let users search products by name.
- Let users filter products to show: All, In Stock only, or Out of Stock only (and have it work together with search, if you built both).
- Show a clear message when a search/filter combination matches nothing.
- Add more summary numbers (e.g. how many are in stock, or the total value of the inventory).
- Let users edit an existing product's name or price.

Do not attempt anything requiring a concept, array method, or browser API you haven't been taught yet — if you're unsure whether something is "allowed," ask yourselves whether it was shown in class through Session 09.

Visual / CSS expectations

CSS is part of the work this time — you are not given a stylesheet. Your dashboard should have:

- A clear page header.
- A visible area for the summary numbers.
- A usable, readable form for adding a product.
- Products displayed in a way that's easy to scan (a list or a grid of cards — your choice).
- Clear visual distinction between in-stock and out-of-stock products.
- Buttons and inputs that look intentional, not like unstyled browser defaults.

- A layout that stays usable at a narrower browser width.

You decide the exact look. There is no reference design to match.

Testing error behavior

To see your error state and Retry actually work, temporarily rename `products.json` (so your app's fetch gets a "not found" response instead of real data), reload, then rename it back and use Retry to confirm recovery. Note: a "not found" response for a JSON file typically isn't valid JSON itself — trying to read it as JSON is what will trigger your error handling, not the request itself "failing." Simply pointing at a different but still-real, still-valid-JSON file would NOT trigger your error path — think about why.

Allowed concept boundary

Allowed: everything taught through Session 09 — components, typed props, `useState`, controlled inputs, event handlers, conditional rendering, `.map()`, `.filter()`, `.reduce()`, derived data, callback props, collection state, immutable array/object updates, `useEffect` with an empty dependency array, `async / await`, `try / catch`, loading/error/Retry.

Not yet — do not use: `useEffect` with anything other than an empty dependency array, props destructuring, React Fragment, a separate shared types file, any array method you haven't been taught, `localStorage`, routing, Context, custom hooks, or any state-management library.

Acceptance criteria (CORE)

- ☐ The app fetches real data from `products.json` on load, with a visible loading state.
- ☐ A broken fetch shows a clear error message with a working way to retry.
- ☐ Every product's name, category, price, and stock status are visible.
- ☐ At least one summary number is visible and updates correctly as the product list changes.
- ☐ A new product can be added and immediately appears in the list.
- ☐ A product's stock status can be toggled.
- ☐ A product can be removed.
- ☐ The interface is coherent, readable, and usable at a narrower width.
- ☐ `npm run build` passes with no errors.

Optional extras you attempt should also work correctly and not break anything above, but they are not required for a complete, successful CORE submission.

Team plan (do this first, before writing code)

As a team, briefly answer:

- What pieces of information does this app need to keep track of, and which of them can CHANGE over time?
- What are the main visual sections your dashboard will need?
- Which numbers on the dashboard are calculated FROM other data, rather than being their own piece of information?
- How will your team split the initial work?

Team reflection (do this last)

As a team, briefly answer:

- Which information did your app store as state, and why?
- Which values did you calculate from other data instead of storing separately?
- What components did your team create, and what was each one responsible for?
- Where did an action in a child component need to affect something in a parent component?
- What was your hardest bug, and how did you find it?
- What would you improve with 30 more minutes?